DYNAMIC TRAFFIC SIGNAL CONTROL USING REINFORCEMENT LEARNING

A Project Report

submitted by

AKASH ANKUSH KAMBLE

in partial fulfillment of the requirements
for the award of the degree of

MASTER OF TECHNOLOGY



DEPARTMENT OF COMPUTER SCIENCE
AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS.

April 2019

THESIS CERTIFICATE

This is to certify that the thesis titled **DYNAMIC TRAFFIC SIGNAL CONTROL USING REINFORCEMENT LEARNING**, submitted by **Akash Ankush Kamble**, to the Indian Institute of Technology, Madras, for the award of the degree of **Master of Technology**, is a bona fide record of the research work done by him under our supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Dr. Prashanth L. A.
Research Guide
Assistant Professor
Dept. of Computer Science and Engineering
IIT-Madras, 600 036

Place: Chennai

Date: April 2019

ACKNOWLEDGEMENTS

I want to thank several individuals who in one or another way contributed and extended their valuable assistance throughout this project.

I would like to sincerely thank my guide, Dr. L. A. Prashanth for his unique ideas, critical suggestions and visionary directions throughout the project. His humbleness, passion, simplicity, and experience has been a constant source of inspiration. I thank my friends for helping me in my work and suggesting improvements.

I owe a debt of gratitude to the Indian Institute of Technology, Madras for giving me such an opportunity to work with some of the brightest minds in the country. Finally I would like to thank my parents who are my pillars of strength in every step of my life.

ABSTRACT

KEYWORDS: reinforcement learning (RL), traffic signal control (TSC), Monte Carlo control (MCC), Q-learning, function approximation (FA).

We implement three reinforcement learning (RL) algorithms for dynamic traffic signal control. The Q-learning-based algorithm, QLFA includes state-action features whereas the Monte Carlo control and the QLFS algorithms require full state representations. The necessary quantity of features is the number of incoming lanes towards the junction where the signal is, and the amount of elapsed time since the last switch of signal from 'red' to 'green' or vice versa. We implement these dynamic algorithms in three settings and compare their performance with other non-dynamic (fixed controller) algorithms.

We observe that the dynamic algorithms perform better than their other fixed controller counterparts on the road network settings we have considered. Among the two Q-learning algorithms, the function-approximation-based variant performs better than full-state-representation-based one.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS i

ABSTRACT ii

LIST OF TABLES v

LIST OF FIGURES vi

ABBREVIATIONS vii

NOTATION viii

1 INTRODUCTION 1

2 The Traffic Signal Control Problem 3

2.1 MDP structure 3

2.2 Traffic Signal Control problem as an MDP 7

3 RL Algorithms 10

3.1 Monte Carlo Control (MCC) 10

3.2 Q-learning with full state representations (QLFS) 11

3.3 Q-learning with feature-based representations (QLFA) 12

3.3.1 Feature-Based Representations 12

3.3.2 Q-learning with Function approximation 14

4 Simulation experiments: full-state-based representations 16

4.1 Experimental Setup 17

4.2 Results 18

4.2.1 Average waiting Time 18

4.2.2 Total Number of Vehicles (or Road Users) 19

5 Simulation experiments: feature-based representations 22

5.1 Experimental Setup 22
5.1.1 Scenario (1): two crossroads with a single junction 22
5.1.2 Scenario (2): two-junction corridor 24
5.1.3 Scenario (3): 2×2 grid network 25
5.2 Results 26
5.2.1 Average Waiting Time (AWT) 26
5.2.2 Total number of vehicles (or road users) (TNV) 28

6 Conclusions **31**

LIST OF TABLES

2.1 Reward distribution 8

LIST OF FIGURES

2.1 The agent-environment interrelationship in a MDP. 3

4.1 SUMO - Graphical User Interface 16
4.2 AWT for MCC 18
4.3 AWT for QLFS 19
4.4 TNV for MCC 20
4.5 TNV for QLFS 20

5.1 Single junction 23
5.2 Two-junction corridor 24
5.3 2×2 grid 25
5.4 AWT for single junction 26
5.5 AWT for two-juncion corridor 27
5.6 AWT for 2×2 grid 27
5.7 TNV for single junction 28
5.8 TNV for two-juncion corridor 29
5.9 TNV for 2×2 grid 29

ABBREVIATIONS

RL	Reinforcement Learning
TSC	Traffic Signal Control
MDP	Markov Decision Process
MCC	Monte Carlo control
FA	Function Approximation
TWT	Total Waiting Time
AWT	Average Waiting Time
TNV	Total Number of Vehicles
ANV	Average Number of Vehicles
QLFS	Q-learning using full-state description
QLFA	Q-learning based on function approximation

NOTATION

$S, \mathcal{S}$	Set of states
$A, \mathcal{A}$	Set of actions
$R, \mathcal{R}$	Set of rewards
s	state
a	action
r	reward
G	expected return
$P, \mathcal{P}$	state transition probability function
π	policy
V, v	state-value function
Q, q	action-value function
α	step size, learning rate
γ	discount factor
$\phi(s, a)$	feature vector ϕ which corresponds to state-action tuple (s, a)
θ	a tunable parameter for QLFA
Φ	state-action feature matrix

CHAPTER 1

INTRODUCTION

The forever decreasing construction space and the steadily growing amount of traffic in all urban and a majority of semi-urban territories demand for a good traffic signal control plan. The traffic junctions decide the blockage condition of the street because these junctions become the main bottleneck for the traffic flow. A majority of traffic signals at such junctions worldwide use phase cycles of fixed time in a round-robin way. Albeit modeling such a system is straightforward, it does not explain the real situation of traffic and often becomes the reason for a much worse congestion problem.

In this project, we examine the adoption of reinforcement learning (RL)-based traffic signal management framework. Our target is dealing with the traffic congestion at the road junctions in such a manner that the overall flow of traffic is maximized and the average delay for the vehicles or road users is lesser than the normal fixed-cycle signal control framework. The vehicle detectors, which are set along the paths that lead to an intersection, send the percentage occupancy of that lane to the main controller or agent which decides the action to carry out depending on this data. The action being, to determine the signal duration, in this case.

A similar problem has been studied in (Prashanth and Bhatnagar, 2011), where the authors propose a Q-learning based algorithm for TSC. However, actions that we formulate here, relate to phase duration, whereas the traffic lights could go green in an arbitrary order there. Our action selection is more natural and road-user friendly since the signal lights do not go green in a random order. Also, there is no direct shift from traffic light being red and then green thereafter. There is an intermediate 'yellow' phase between the red and green phase which allows vehicles in simulation to decelerate just like in real life.

Our contributions are specified below.

- For this problem, we first formulate the TSC Markov decision process (MDP) with round robin phases of traffic lights and a discretized set of duration values as actions.

- We implement Monte Carlo control (MCC) and two Q-learning algorithms using the SUMO simulator(, DLR). The latter is a popular microscopic traffic simulator.
- We examine the performance of these methods or algorithms on the road network setups having a single four-way intersection or crossroads, a two crossroad-junction corridor and a 2×2 four-junction grid. We compare their performance with some pre-timed controller algorithms.

The rest of the report is structured as follows: The immediate next section, section 2, deals with the problem formulation with Markov decision processes (MDPs). In Section 3, we first describe the Monte Carlo method for control and then the Q-Learning algorithm. In Section 4, we discuss our experimental setup and the simulation experiments done with SUMO. In Section 5, we present results for MCC and QLFS algorithms. In section 6, we present our results and observations for the QLFA algorithm based on simulation experiments done. Subsequently, in Section 7, we present the concluding statements.

CHAPTER 2

The Traffic Signal Control Problem

Looking at the problem traffic signal control, we try to find an optimal duration for the signal phases at road intersections to maximize the traffic flow. A 'signal phase' consists of the set of signals that switch simultaneously.

We work out this signal management problem as an MDP with the presumption that the choices are carried out by a central authority, the agent. The MDP formulation of a problem demands the representation of states, actions, rewards and also the target. The fundamental MDP structure is explained below.

2.1 MDP structure

The State, Action and Reward: Here is a figure depicting the agent-environment interaction in an MDP redrawn from Section 3.1 of (Sutton and Barto, 2018).

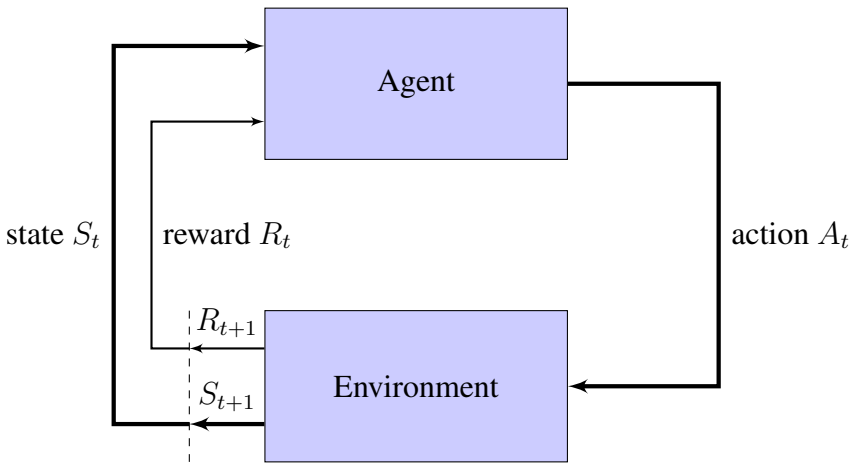


Figure 2.1: The agent-environment interrelationship in a MDP.

"The learner and decision maker is called the $agent$. The thing it interacts with, comprising everything outside the agent, is called the $environment$".(Sutton and Barto, 2018)

The environment also gives incentives or feedback, through the agent's selection of actions, in a form of scalar values, called as 'rewards'. The goal of the agent is to maximize these rewards over time. This time period has a number of distinct time steps, $t = 1, 2, 3, \ldots$, at each of which the agent and the environment interact with each other. At each time step t, the agent is able to get a sense of the current $state$ S_t of the environment, $S_t \in \mathcal{S}$, and based on this information, the agent then chooses an $action$ $A_t \in \mathcal{A}(s)$. As a result of this action, the agent acquires a scalar $reward$ from the environment, $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$, and moves to a different state, S_{t+1}. These states, actions and rewards form a sequence, often called a $trajectory$ which looks like this:

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots$$

Note that theoretically, the sets of states ($\mathcal{S}$), actions ($\mathcal{A}$) and rewards ($\mathcal{R}$), all can have an infinite cardinality, but most of the times, for the purpose of practicality and simplicity, they have a finite cardinality. Such an MDP is termed as finite MDP.

The Markov property: The Markov property is the memory-lessness property of stochastic processes. It states that, "the future is conditionally independent of the past given the present". When the current state in known, the historical data stored so far might be discarded, and that state is an adequate measurement that gives us indistinguishable characterization of the future as if we have all the history (Wiering and van Otterlo, 2014). A state S_t has Markov property *iff*;

$$P[S_{t+1}|S_t] = P[S_{t+1}|S_1, \ldots, S_t] \tag{2.1}$$

A Markov state S and its next state S', have a certain probability associated with them for state transition, which as a function, is defined by,

$$\mathcal{P}_{ss'} = \mathbb{P}[S_{t+1} = s'|S_t = s] \tag{2.2}$$

It is a probability distribution over next possible successor states, given current state.

A Markov process: A memory-less random process or a progression of arbitrary states $S_1, S_2, \ldots$ with the Markov property is called as a Markov process. This process forms a chain of states and is often called a 'Markov chain' or an 'episode'. It is a tuple (S, P) on state space S, and the transition function P.

Markov Decision Process (MDP): A Markov Process with rewards and decisions associated with it, is called as Markov decision process or MDP. It is a 5-tuple (S, A, P, R, γ), where S is the state space, A is action space, P is the state transition probability function,

$$\mathcal{P}^a_{ss'} = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a] \tag{2.3}$$

R is the reward function,

$$\mathcal{R}^a_s = \mathbb{E}[R_{t+1} | S_t =, A_t = a] \tag{2.4}$$

and γ is the discount factor, $\gamma \in [0, 1]$.

There is a notion of the 'expected return', G_t, which is described as some function of the reward chain, which we want to maximize. G_t can be given as follows.

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \tag{2.5}$$

where γ is the discount factor and $\gamma \in [0, 1]$.

The Bellman Equation: A policy π, dictates what action to take based on which state the agent is in. Means, it fully defines the behaviour of an agent. π is given as,

$$\pi(a|s) = \mathbb{P}[A_t = a | S_t = s] \tag{2.6}$$

There is a probability associated with taking action a under policy π from state s. Just like the transition probabilities above, same thing happens with the reward function,

$$\mathcal{R}^\pi_s = \sum_{a \in \mathcal{A}} \pi(a|s) \mathcal{R}^a_s \tag{2.7}$$

The state-value function, which is the expected value of return, given that we start from state S and then follow the policy π, can be represented using the immediate reward R_{t+1}, and discounted value of the next state $\gamma V_\pi(S_{t+1})$ with policy π,

$$v_\pi(s) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s] \tag{2.8}$$

Likewise, the action-value function, which is the expected value of return, given that we start from state S, take action A, and follow the policy π, can be represented as,

$$q_\pi(s,a) = \mathbb{E}_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \tag{2.9}$$

The Bellman expectation equation for $V_\pi(s)$ turns out to be,

$$V_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left(\mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v_\pi(s') \right) \tag{2.10}$$

and the Bellman expectation equation for $q_\pi(s,a)$ becomes,

$$q_\pi(s,a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \sum_{a' \in \mathcal{A}} \pi(a'|s') q_\pi(s', a') \tag{2.11}$$

Optimal Value Functions: The maximum state-value function from all policies should be the optimal state-value function $V^*(S)$.

$$v^*(s) \doteq \max_\pi v_\pi(s)$$

Likewise, the action-value function that gives maximum value from all policies should be the optimal one.

$$q^*(s,a) \doteq \max_\pi q_\pi(s,a)$$

This value will tell us the best action to choose amongst all actions regardless of the state or the policy.

The Optimal Policy: We can get the optimal policy to pursue, using the maximization over $q^*(s,a)$. This will modify the policy's state-action mapping in such a way that will allow the agent to pick best action every time.

$$\pi^*(a|s) = \begin{cases} 1 & \text{, if } a = \arg\max_{a \in \mathcal{A}} q^*(s,a) \\ 0 & \text{, otherwise} \end{cases} \tag{2.12}$$

Bellman Optimality Equations: With the above equations we can derive Bellman

Optimality equation for $V^*(S)$ as,

$$v^*(s) = \max_a q^*(s, a)$$

and, the Bellman Optimality equation for $Q^*(S, A)$ as,

$$q^*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v^*(s')$$

Finally, a recursive Bellman Optimality equation for $V^*(s)$ can be written as,

$$v^*(s) = \max_a \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a v^*(s') \tag{2.13}$$

and the recursive Bellman Optimality equation for $Q^*(s, a)$ can be written as,

$$q^*(s, a) = \mathcal{R}_s^a + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}_{ss'}^a \max_{a'} q^*(s', a') \tag{2.14}$$

Solving these optimality equations will make the agent follow the optimal policy and choose the best possible action every time.

2.2 Traffic Signal Control problem as an MDP

The problem for the crossroad networks having a single junction, a two crossroad-junction passage and a 2×2 four-junction grid is being formulated in this section. The intersection has two crossroads, each with two lanes. For all crossroads, out of its two lanes, one of them is an incoming lane towards the intersection. The dynamic algorithms need a specification of states, action, and rewards.

State: If the total number of incoming lanes towards the junction is n, then for the MCC and QLFS algorithm, the state is a n dimensional vector of queue lengths, categorized into three traffic categories, 1) low, 2) medium and 3) high, depending on the queue length for each of the crossroad lane coming towards the junction. In that case the state at time t is for the junction or agent is,

$$s_t = (q_1(t), q_2(t), \ldots, q_n(t))$$

where $q_i(t)$ is the length of the queue on the ith lane at time t.

For the QLFA algorithm, we normalize those uncategorized queue lengths into a single i^{th} feature value belonging to the interval $(-1, 0]$, as shown.

$$f_i = \frac{q_1(t) + q_2(t) + \cdots + q_n(t)}{n}$$

We take one more feature to represent the state for QLFA, and that is the elapsed time since the last switch of the signal. We normalize it to be in the $(-1, 0]$. So, the feature vector for an action a in a state s at time t is,

$$\phi(s, a_i)_t = (\phi(s, a_i)(1)_t, \phi(s, a_i)(2)_t, \ldots, \phi(s, a_i)(f)_t)^T$$

Phase changes to the traffic light are carried out by the regulator that accepts the traffic state of the incoming lanes and takes decisions on for how long a signal phase is to be kept at 'red' (or 'green' for the other phase), based on the state information. This decision is conveyed to the junction.

Action: The set of actions, a, holds the available choices of actions for all phases,

$$a = ((d_1^1, d_2^1, \ldots, d_k^1), (d_1^2, d_2^2, \ldots, d_k^2), \ldots, (d_1^n, d_2^n, \ldots, d_k^n))$$

where d_j^i is the jth available duration choice for the ith traffic light phase.

Reward: For the full-state-based as well as fixed controller algorithms, the reward function has a single component which is the sum of individual lane-rewards based on the queue lengths of those lanes. Each queue length is associated with one of the categories (low, medium and high) and is given an immediate reward. The rewards for each category are given as shown below in table (2.1).

Lane occupancy percentage	Traffic category	Reward
0 - 29	Low	0
30 - 59	Medium	-1
60 - 100	High	-2

Table 2.1: Reward distribution

All individual rewards are then summed up to get total reward of a particular action at a particular state.

$$r(s_t, a_t) = r_1 + r_2 + \cdots + r_n$$

where, n is the number of lanes coming towards the junction. Four in our case. For each lane, the rewards are categorized into three categories, 1) low, 2) medium, and 3) high. These are determined using the occupancy percentage of each lane using queue length and the total length of the lane.

For the feature-based algorithm, the reward has two components which are the queue length of the lanes and the time elapsed since the last toggle by the traffic signal respectively. Both the components are normalized to be real numbers between 0 and -1. Individual rewards are then summed up to get total reward of a particular action at a particular state.

$$r(s_t, a_t) = r_l + r_t$$

where, r_l is the length feature and r_t is the time feature.

CHAPTER 3

RL Algorithms

3.1 Monte Carlo Control (MCC)

We recall the MCC algorithm below (see Section 5.4 of (Sutton and Barto, 2018) for a detailed description).

Input: Algorithm parameter: $\epsilon > 0$, episode length: T

Initialization:

$\pi \leftarrow$ an arbitrary policy

$Q(s,a) \leftarrow random\ numbers \in (0,1]$ (arbitrarily), $\forall s \in \mathcal{S}, \forall a \in \mathcal{A}$

$Returns(s,a) \leftarrow$ empty table, $\forall s \in \mathcal{S}, \forall a \in \mathcal{A}$

Repeat Forever (for all episodes)**:**

Produce an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop (for all steps of the current episode, $t = T-1, T-2, \ldots, 0$):

$G \leftarrow G + R_{t+1}$

If the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \ldots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow$ average($Returns(S_t, A_t)$)

$A^* \leftarrow \arg\max_a Q(S_t, a)$

For all $a \in \mathcal{A}(S_t)$:

$$\pi^*(a|S_t) \leftarrow \begin{cases} 1-\epsilon & \text{, if } a = A^* \\ \epsilon & \text{, if } a \neq A^* \end{cases}$$

Algorithm 1: Monte Carlo Control (MCC)

The first algorithm we implement for traffic signal control is this one. In algorithm (1), we fix an arbitrary ϵ-soft policy and assign Q values randomly for all states-action

pairs. We also prepare a $Returns$ list for those pairs. We then capture the state-action-reward sequence for all steps of the learning cycle. For each step of the episode, we then keep collecting immediate cumulative reward in reverse order of steps and look out for state-action pair of that step. If that particular pair appears in the episode somewhere, we store that cumulative reward in $Returns$ list for that specific pair and for Q value of that pair we assign an average of this return. Next, we determine the best action for this particular state and updates the policy π according to its rule.

3.2 Q-learning with full state representations (QLFS)

We recall the Q-learning algorithm below (see Section 6.5 of (Sutton and Barto, 2018) for a detailed description).

Input: Parameters for algorithm: $\alpha \in (0, 1]$, $\epsilon > 0$, episode length: T

Initialization:

$Q(s, a) \leftarrow random\ numbers \in (0, 1]$ (arbitrarily), $\forall s \in \mathcal{S}^+$, $\forall a \in \mathcal{A}$

Loop (for all episodes)**:**

Initialize S

Loop (for all steps of the current episode):

Choose A from S using ϵ-greedy policy derived from Q

Take action A, observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

Algorithm 2: Q-learning

We implement the second algorithm, the Q-learning algorithm (2), for controlling the traffic signal. This algorithm updates Q values at each step according to its update rule given in Section 6.5 of (Sutton and Barto, 2018),

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

With this update rule, we directly estimate the optimal action-value function, Q^*, using

the assimilated action-value function, regardless of the policy followed. Still, the policy decides which state-action pairs the agent visits and updates it.

3.3 Q-learning with feature-based representations (QLFA)

In the previous chapter, we have represented the value functions by a look-up table where every state S has an entry $V(s)$ or every state-action pair (s, a) has an entry $Q(s, a)$. For each iteration, we perform updates on our values V_i^* until convergence. Each iteration, we can also update the policy π_i^* for each state.

However, in practice due to the curse of dimensionality, a look-up table based algorithm does not scale. There are just too many states and/or actions to store in memory, for a large MDP. When an RL problem has a huge state-action space, we can not represent V or Q functions in tabular format, regardless of the amount of memory we have. To overcome the curse of dimensionality, a standard approach is to incorporate feature-based representations for solving large MDPs.

3.3.1 Feature-Based Representations

Because we will never have enough training data in cases of large state spaces, we need to derive what we have grasped from one state to other 'matching' states. The objective is to use a parameterized function, as opposed to utilizing a large table to store V and Q-values such that the number of parameters is a few orders of magnitude smaller than the number of states. The learning of parameters will be done from experience. This way, when we update the parameters based on examinations in one state, then our V and Q estimates will also change for other similar states, that is, "the parameterization facilitates generalization of experience". There are many function approximators, e.g.

- Linear Combinations of features,
- Neural networks,
- Decision trees,
- Nearest neighbours, etc.

We consider the first approximator mentioned above, which is to employ a linear combination of features to approximate the Q-values, $Q(s,a)\ \forall(s,a)$. Note that, Q-values are used to choose actions as follows.

$$\pi(s) = \arg\max_{a} Q^*(s,a) \tag{3.1}$$

For this we find a set of f features that are functions of states and actions, $\phi_1(s,a)$, $\cdots$, $\phi_f(s,a)$. The features are utilized as our descriptions of states and actions. States with matching feature values will be considered to be similar. We then approximate the $Q(s,a)$ using linear approximation as $Q(s,a) \approx \theta^T\phi(s,a)$ (i.e. weighted sum of the features),

$$Q(s,a) \approx \theta_0 \cdot 1 + \theta_1\phi_1(s,a) + \cdots + \theta_f\phi_f(s,a) = \theta^T\phi(s,a) \tag{3.2}$$

The accuracy of this approximation is basically restricted by the details resolved by the features. Here, $\phi(s,a)$ is an f-dimensional attribute or feature vector which accords with the state-action pair (s,a), where $s \in S$, and $a \in A$. The other component, θ is an adjustable parameter which has the same dimensionality as $\phi(s,a)$.

Given n actions and two extractable features per action, the features for state-action pair (s,a_i) can be encoded as:

$$\phi(s,a_1) = \begin{bmatrix} \psi_1(s,a_1) \\ \psi_2(s,a_1) \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \\ 1 \end{bmatrix}, \phi(s,a_2) = \begin{bmatrix} 0 \\ 0 \\ \psi_1(s,a_2) \\ \psi_2(s,a_2) \\ \vdots \\ 0 \\ 0 \\ 1 \end{bmatrix}, \cdots, \phi(s,a_n) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ \psi_1(s,a_n) \\ \psi_2(s,a_n) \\ 1 \end{bmatrix}$$

Let Φ denote a matrix which has its rows represented as $\phi_{s,a}^T$, where $s \in S, a \in A(s)$. Let

$$\phi_{s,a} = (\phi_{s,a}(1), \ldots, \phi_{s,a}(f))^T.$$

We represent Φ as, $\Phi = (\Phi(j), j = 1, \ldots, f)$, where $\Phi(j)$ is a single vector. Then we

define $\Phi(j)$ as,

$$\Phi(j) = (\phi_{s,a}(j), s \in S, a \in A(s))^T, \qquad j = 1, \ldots, f.$$

We represent the parameter θ as, $\theta = (\theta_1, \ldots, \theta_f)^T$. Then

$$Q \approx \sum_{i=1}^{f} \Phi(i)\theta_i, \qquad \text{or instead, } Q \approx \Phi\theta.$$

Here Q is the Q-values vector $Q(s, a)$ for all fitting (s, a) pairs, so $Q = [Q(s, a)|s \in S, a \in A(s)]^T$. To put it another way, we approximated Q using $\Phi\theta$. The gradient of the now-approximated Q function with respect to θ is,

$$\nabla_\theta Q(s, a) \approx \phi_{s,a}.$$

The dimension (f) of the feature vector is minuscule in comparison with the size of the state-action space, e.g., in a four-junction grid network, the size of the state-action space is of the order of 3^{16}, in contrast, the size of f is only 13.

3.3.2 Q-learning with Function approximation

Here, we divulge the feature-based TSC algorithm. The full-state-based algorithm, QLFS, needs entire state-space data and is computationally sub-par, its feature-based variation parameterizes the action-value function and warrants considerably reduced calculation concerning space and time demands while performing excellently. This QLFA algorithm updates the tunable parameter θ, which is an f-dimensional amount. In this manner, rather than resolve a system which has $|S \times A(s)|$ variables, we solve here it using just f factors. As previously mentioned, on account of the four-junction grid that we study in our examinations, it has appeared, even though $|S \times A(s)| \sim 3^{16}$, the f is just 13. This results in major reduction in the computation time if feature-based models are utilized. The QLFA algorithm updates θ with this rule shown hereunder:

$$\theta \leftarrow \theta + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \vec{\nabla}_\theta Q(s, a) \tag{3.3}$$

where,

- $\alpha \in [0, 1]$ is the learning rate or step size,
- $\gamma \in [0, 1]$ is the discount factor,
- $\max_{a'} Q(s', a')$ is the action a' in the state s' maximizing $Q(s', a)$,
- $\vec{\nabla}_\theta Q(s, a)$ is the gradient of $Q(s, a)$ in θ. In our linear case, the gradient is simply $\sum_{i=1}^{f} \phi_i(s, a)$.

This parameter update rule tells us that $[R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ is the error between predicted value $Q(s, a)$ and the real $Q(s, a)$, which is the reward r obtained now plus the expected, discounted reward following the greedy policy afterwards $\gamma * \max_{a'} Q(s', a')$. This shows that the parameter is shifted into the steepest direction (given by the gradient $\vec{\nabla}_\theta Q(s, a)$) by the amount of the measured error, adjusted by α.

CHAPTER 4

Simulation experiments: full-state-based representations

For all the simulation experiments in this project, we use a traffic simulator software named '**S**imulation of **U**rban **MO**bility' or SUMO for short. It is an open source, microscopic road traffic simulation package and it is licensed under Eclipse Public License - v 2.0 (DLR). Here is a snapshot of its interface with the road network scenario, with which the experiments have been performed, loaded.

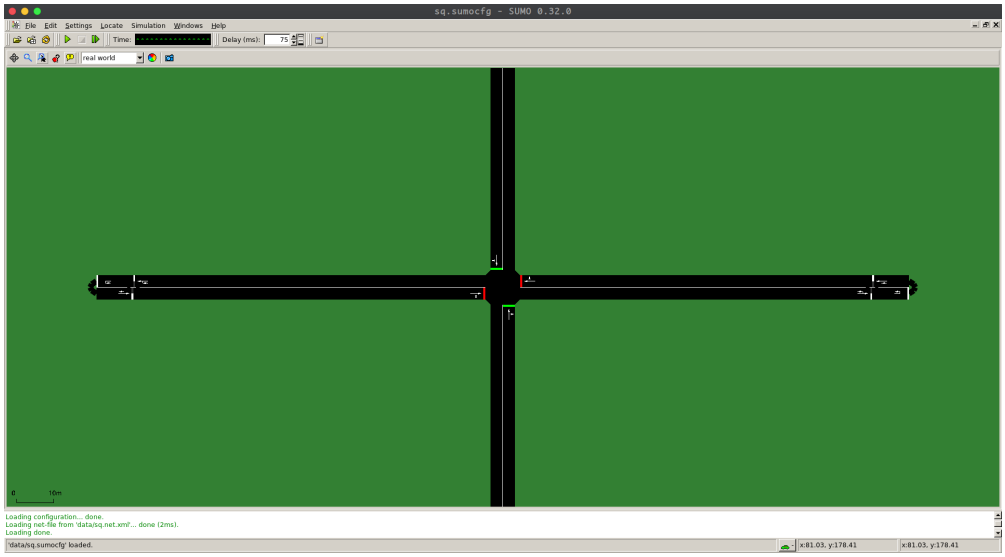


Figure 4.1: SUMO - Graphical User Interface

To control the traffic during simulation runtime, we use an extension to SUMO, called '**Tra**ffic **C**ontrol **I**nterface' or TraCI. It is a library of commands and methods which provide value retrieval, changing state, subscriptions, and accessing generic parameters. TraCI offers access to running street traffic simulation, and it also permits to recover attributes of simulated objects and to manipulate their actions online. It utilizes a TCP based client/server design to give access to SUMO. TraCI interfaces are available for multiple programming languages, e.g., Python, Java, .NET, MATLAB and C++, of which, we have used the one available for Python because it is tested daily and supports all TraCI commands (DLR).

4.1 Experimental Setup

We use the following setup for our simulation experiments. Along with the dynamic algorithms, we also run four other non-dynamic, fixed-time controller algorithms with the north-south/east-west phase duration 20/20, 25/25, 20/25 and 25/20 seconds in each experiment. For both, the MCC and the QLFS algorithm, the episode length T is set to 850 time steps. Also the exploration factor ϵ is set to 0.2 for both ($0 \leq \epsilon < 1$), which means that for 20% of the times the agent will not select the best action available to it, but it will choose a random action from the actions set. For the rest of the times, it will select the best action available. For the Q-learning algorithm, we use discounted rewards. For that, we have chosen the discount factor γ to be 0.9 ($0 \leq \gamma \leq 1$). The Q-learning algorithm also needs one more parameter which is the step size α. Step size is also called as 'learning rate.' It determines the amount of change in Q-values during an update. We set the α to be 0.05 ($0 < \alpha \leq 1$). For both the algorithms, the initialization of the Q-values matrix is the same. We populate it with random real numbers that belong to the interval $(0, 1]$. So, the hyperparameters for this experiment are as follows:

- $T = 850$ for both, MCC and QLFS
- $\epsilon = 0.2$
- $\gamma = 0.9$
- $\alpha = 0.05$

The running time of the algorithms in this experiments is somewhere between 12500 and 17500 simulation time steps, however, we load a fixed total of 7942 vehicles each with its vehicle id, route, departure time and the driver imperfection factor, 'sigma' = 0.6 (between 0 and 1). Each vehicle's route was randomly specified using a demand-per-second value such that the vehicle chooses any one of the remaining $(n - 1)$ available lanes. This setup is similar to the real-world traffic situations on a lot of busy intersections. The roads do not have dedicated left turning lanes as of now, but can be included in future.

4.2 Results

We measure the performance of TSC algorithms with following two metrics.

(A) Average Waiting Time (AWT)

(B) Total Number of Vehicles arrived (TNV)

Here, TNV denotes the total number of vehicles that have reached their destination and ANV can be interpreted accordingly.

Let's take a look at the comparisons we made based on (A), and (B).

4.2.1 Average waiting Time

Figures (4.2) and (4.3) present AWT performance of MCC and Q-learning algorithm respectively.

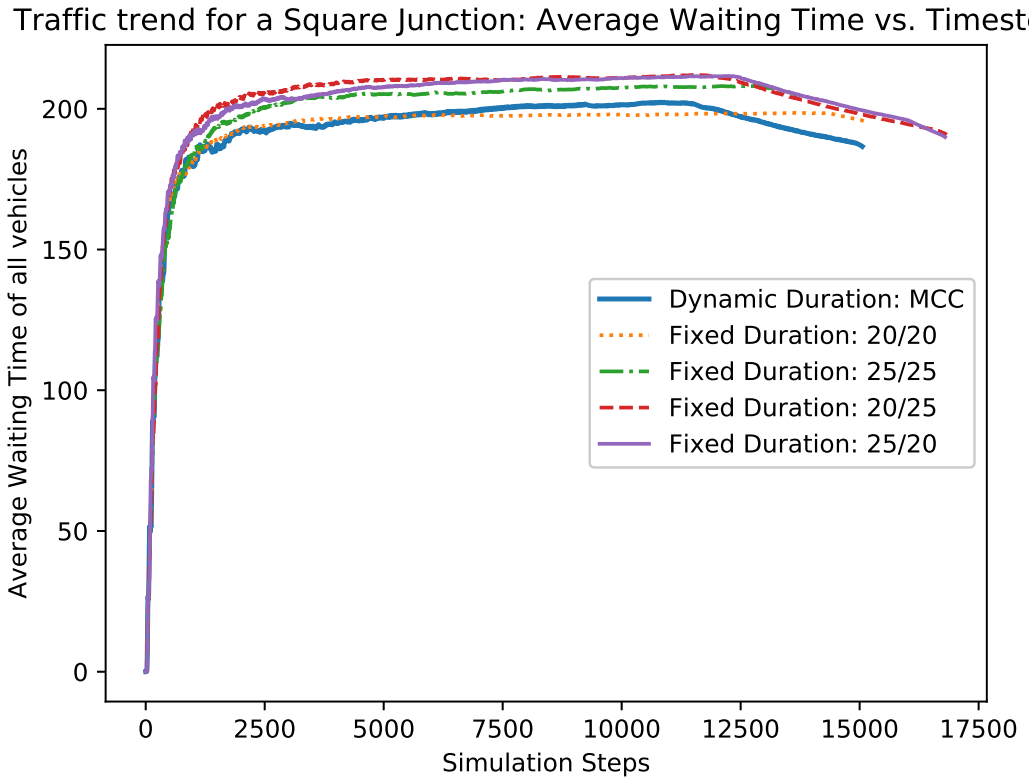


Figure 4.2: AWT for MCC

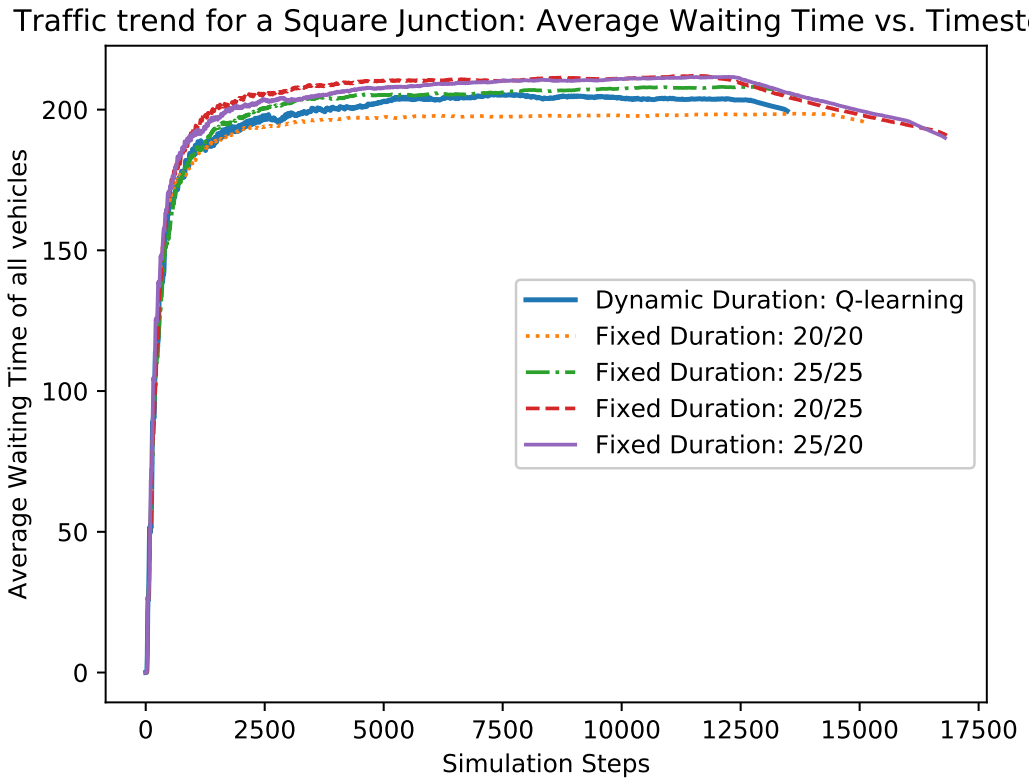


Figure 4.3: AWT for QLFS

From these two figures we observe that MCC algorithm outperforms all other fixed time controller algorithms whereas Q-learning algorithm maintains second best AWT while making the same number of vehicles reach their destination in least time.

4.2.2 Total Number of Vehicles (or Road Users)

Figures (4.4) and (4.5) present TNV performance of MCC and Q-learning algorithm respectively.

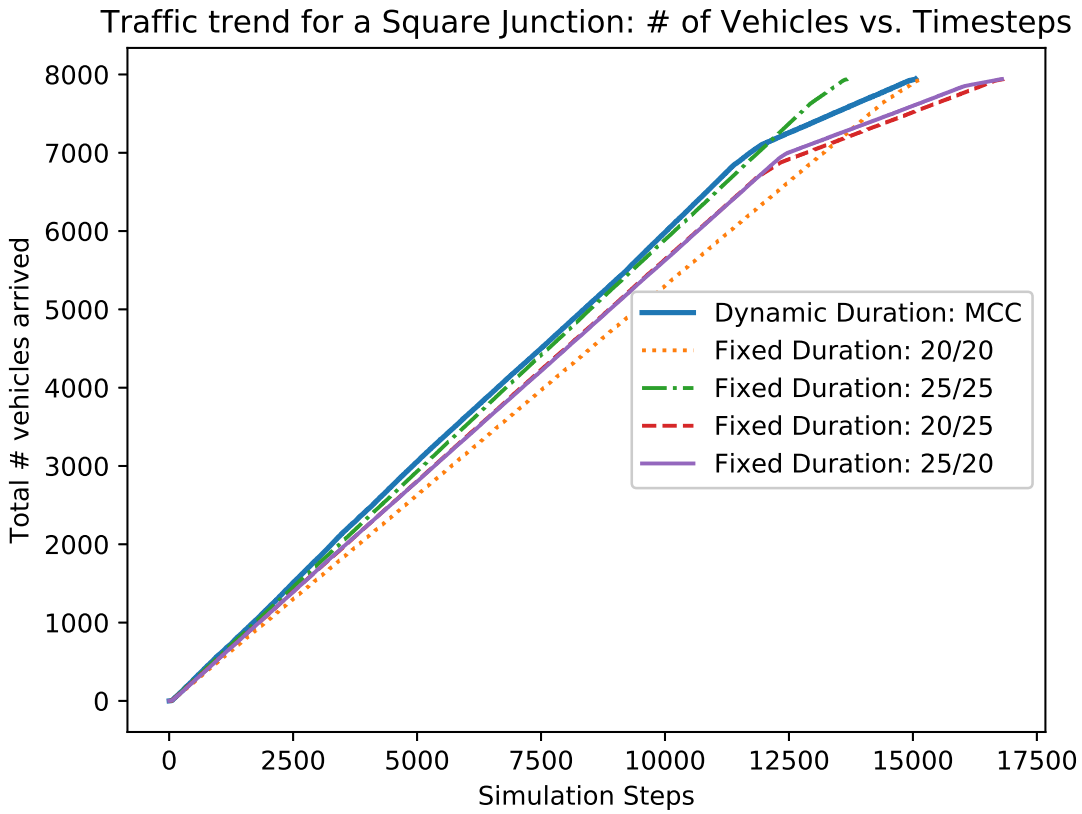


Figure 4.4: TNV for MCC

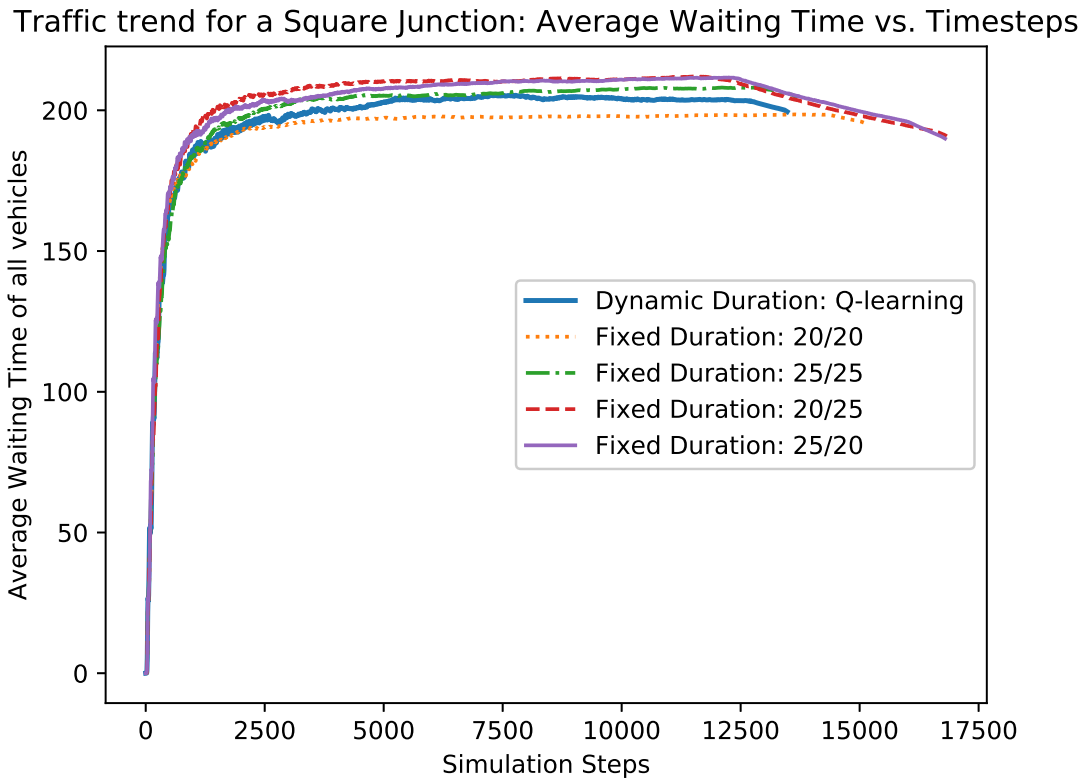


Figure 4.5: TNV for QLFS

From these two figures it can be observed that the Q-learning algorithm is able to send all the vehicles to their destinations within the least amount of time (i.e., time steps) as compared to other fixed cycle controller algorithms. The MCC algorithm gets the second spot here.

This entire analysis shows that dynamic TSC algorithms can and do perform better than traditional fixed time TSC algorithms in reducing the average waiting time while maintaining high traffic flow.

CHAPTER 5

Simulation experiments: feature-based representations

5.1 Experimental Setup

For the feature-based algorithm, we make use of these three different road network scenarios:

1. a simple junction(two crossroads);
2. a two crossroad-junction passage;
3. a 2×2, four-junction grid.

All the roads consist of two lanes, one-way each. Just like the previous experiment, along with the dynamic algorithms we also run the same four other non-dynamic, fixed-time controller algorithms with the north-south/east-west phase duration 20/20, 25/25, 20/25 and 25/20 seconds in each experiment.The road networks are shown in the next three figures, which are snapshots obtained from SUMO simulator GUI. For all the experiments, each vehicle's route was randomly specified using a demand-per-second value such that the vehicle chooses any one of the remaining $(n - 1)$ available lanes. These setups are similar to the real-world traffic situations on a lot of road networks.

5.1.1 Scenario (1): two crossroads with a single junction

Here is a snapshot of SUMO GUI with the road network scenario during simulation.

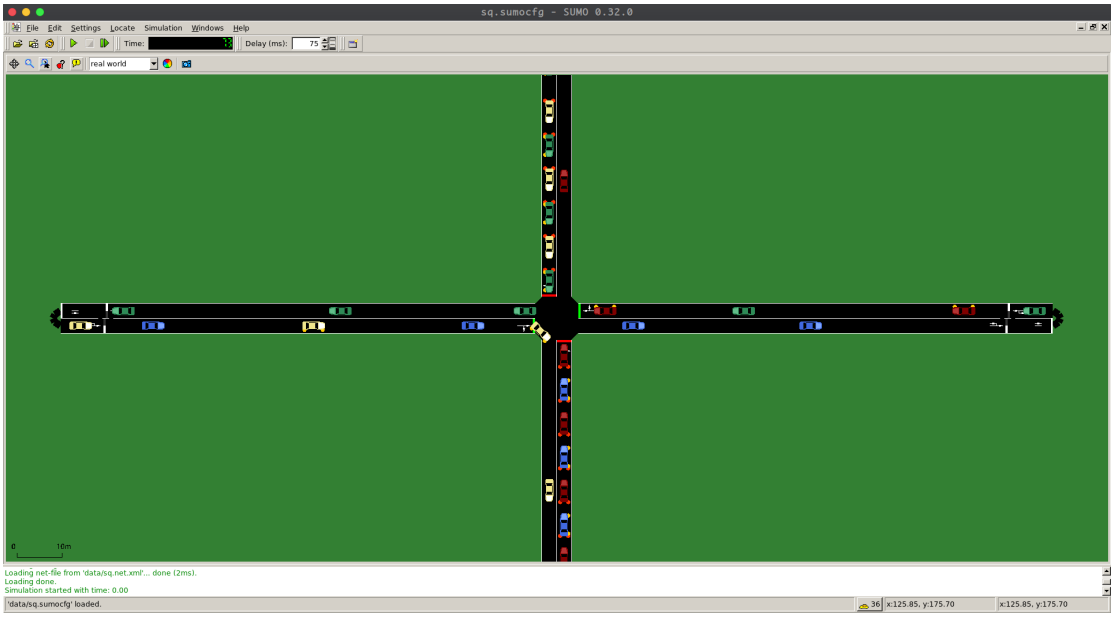


Figure 5.1: Single junction

We use this setup for our simulation experiments. For the QLFA algorithm, the episode length T is set to 75 time steps. Also the exploration factor ϵ is set to 0.2 ($0 \leq \epsilon < 1$). This means that for 20% of the times the agent will not select the best action available to it but it will choose a random action from the actions set. For rest of the times it will select best action available. We also use discounted rewards. For that, we have chosen the discount factor γ to be 0.9 ($0 \leq \gamma \leq 1$). The Q-learning algorithm also needs one more parameter which is the step size α. Step size is also called as learning rate. It determines, the amount of change in Q-values during an update. We set the α to be 0.05 ($0 < \alpha \leq 1$). The initialization of the Φ and $Q(s, a)$ matrix as well as the parameter vector θ, is done by populating it with random real values that belong to the interval $(0, 1]$. The hyperparameters for this experiment with scenario (1) are as follows:

- $T = 75$ for both, QLFS and QLFA
- $\epsilon = 0.2$
- $\gamma = 0.9$
- $\alpha = 0.05$

The running time of the algorithms in this experiments is somewhere between 10000 and 14000 simulation time steps, however, we load a fixed total of 7942 vehicles each with its own vehicle id, route, departure time and the driver imperfection factor, 'sigma' = 0.3 (between 0 and 1).

5.1.2 Scenario (2): two-junction corridor

Here is a snapshot of SUMO GUI with the road network scenario during simulation.

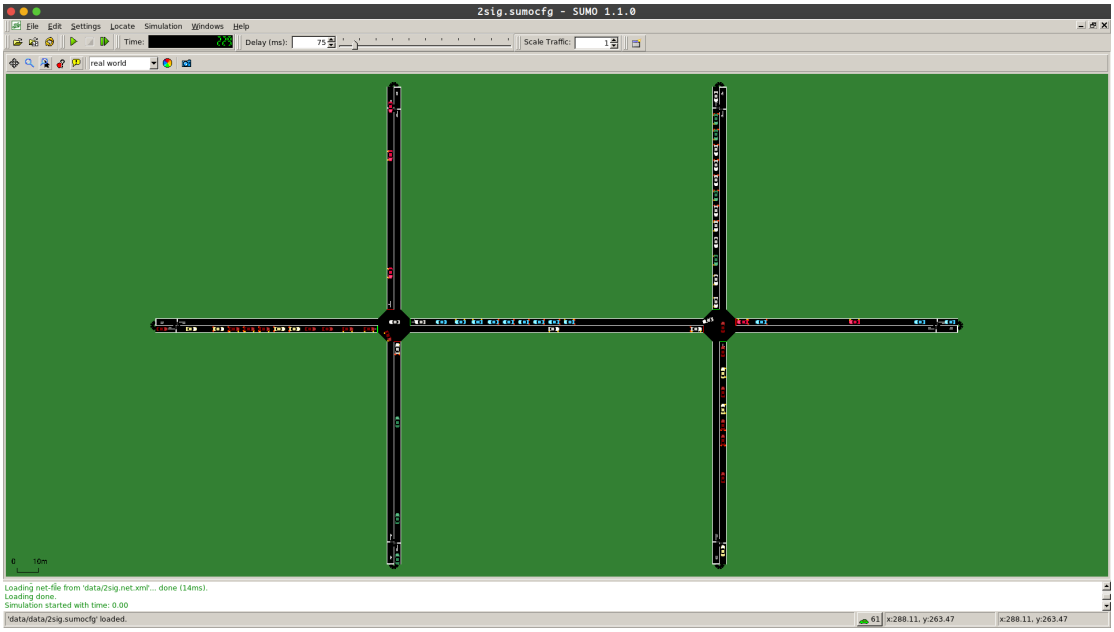


Figure 5.2: Two-junction corridor

For both, the QLFA and QLFS algorithms, the episode length T is set to 300 time steps. Also the exploration factor ϵ is set to 0.2 for both. We have chosen the discount factor γ to be 0.9. We set the α to be 0.05. The initialization of the Φ and $Q(s, a)$ matrix as well as the parameter vector θ, is done by populating it with random real values that belong to the interval $(0, 1]$. The hyperparameters for this experiment with scenario (2) are as follows:

- $T = 300$ for both, QLFS and QLFA
- $\epsilon = 0.2$
- $\gamma = 0.9$
- $\alpha = 0.05$

The running time of the algorithms in this experiments is somewhere between 11000 and 15000 simulation time steps. We load a fixed total of 9951 vehicles each with its own vehicle id, route, departure time and the driver imperfection factor, 'sigma' = 0.3 (sigma $\in [0, 1]$).

5.1.3 Scenario (3): 2×2 grid network

Here is a snapshot of SUMO GUI with the road network scenario during simulation.

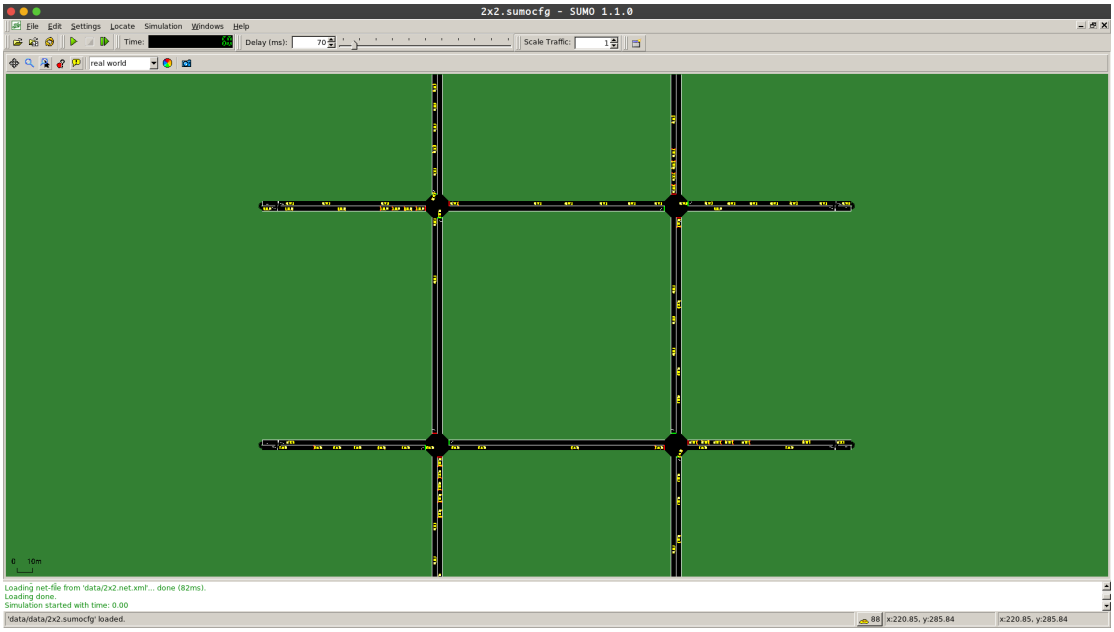


Figure 5.3: 2×2 grid

For both, the QLFA and QLFS algorithms, the episode length T is set to 300 time steps. Also the exploration factor ϵ is set to 0.2 for both. We have chosen the discount factor γ to be 0.9. We set the α to be 0.05 for QLFA and 0.075 for QLFS algorithm. For both the algorithms, the initialization of the Q-values for QLFS and Φ matrix and parameter θ for QLFA, is the same. We populate them with random real numbers that belong to the interval $(0, 1]$. The hyperparameters for this experiment with scenario (3) are as follows:

- $T = 300$ for both, QLFS and QLFA
- $\epsilon = 0.2$
- $\gamma = 0.9$
- $\alpha = 0.05$ for QLFA and $\alpha = 0.075$ for QLFS

The running time of the algorithms in this experiments is somewhere between 18000 and 25000 simulation time steps. We load a fixed total of 22359 vehicles each with its own vehicle id, route, departure time and the driver imperfection factor, 'sigma' = 0.2 (sigma $\in [0, 1]$).

5.2 Results

We measure the performance of TSC algorithms with following two metrics.

1. Average Waiting Time (AWT)
2. Total Number of Vehicles arrived (TNV)

Here, TNV denotes the total number of vehicles that have reached their destination and ANV can be interpreted accordingly. Let's take a look at the comparisons we made based on (1) and (2).

5.2.1 Average Waiting Time (AWT)

Figures (5.4), (5.5) and (5.6) present AWT performance of QLFS and QLFA algorithm respectively on the three scenarios we described earlier.

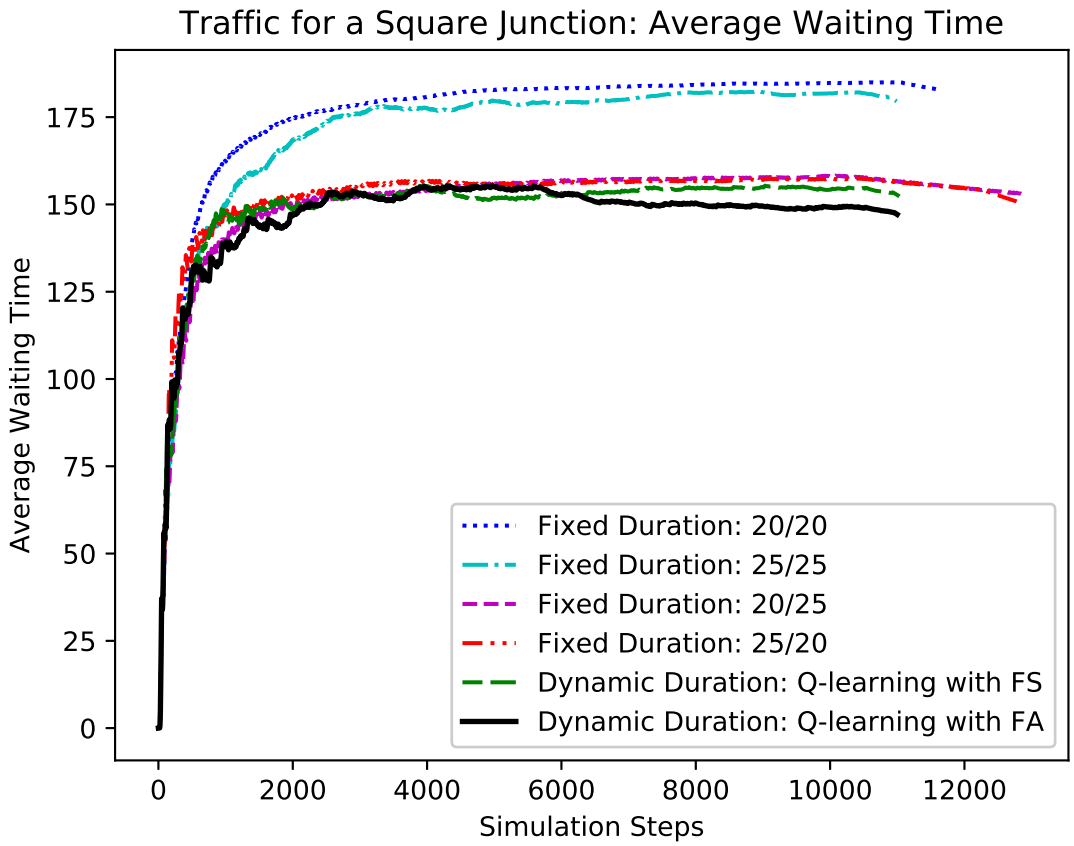


Figure 5.4: AWT for single junction

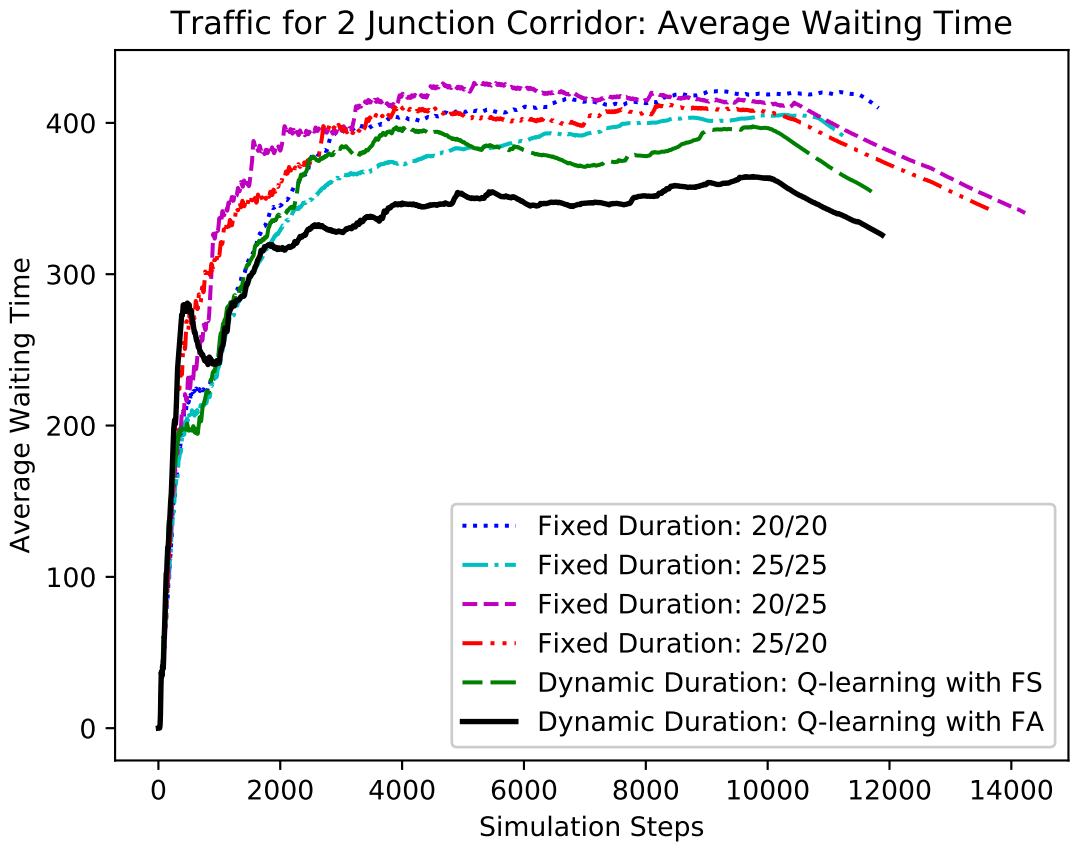


Figure 5.5: AWT for two-juncion corridor

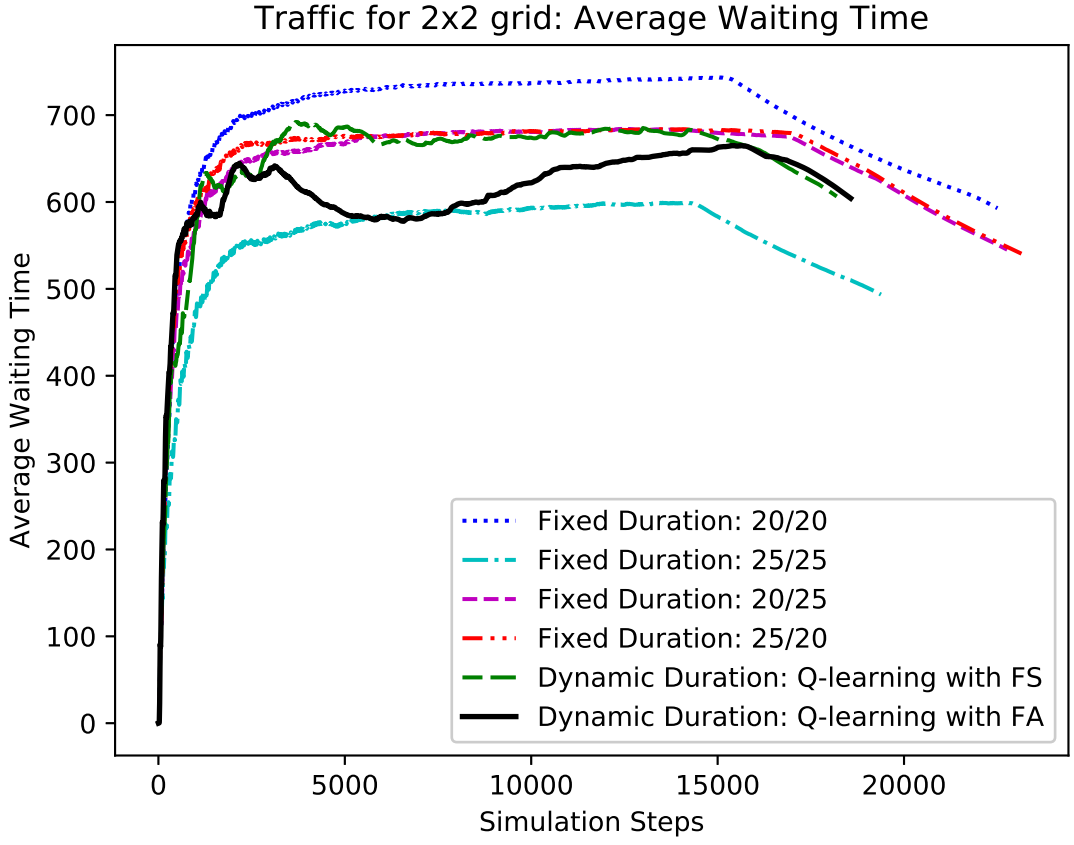


Figure 5.6: AWT for 2×2 grid

From these figures we observe that both of our dynamic duration algorithms perform better than the rest of fixed duration algorithms.

5.2.2 Total number of vehicles (or road users) (TNV)

Figures (5.7), (5.8) and (5.9) present AWT performance of QLFS and QLFA algorithm respectively on the three scenarios we described earlier.

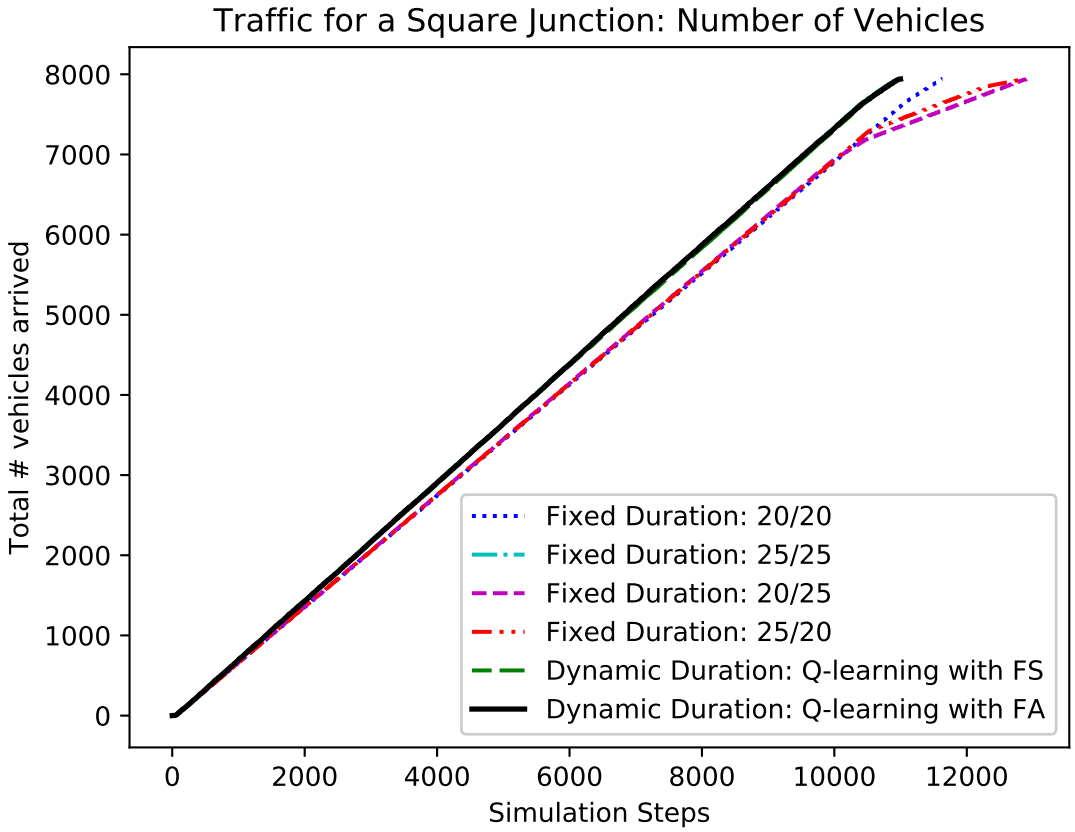


Figure 5.7: TNV for single junction

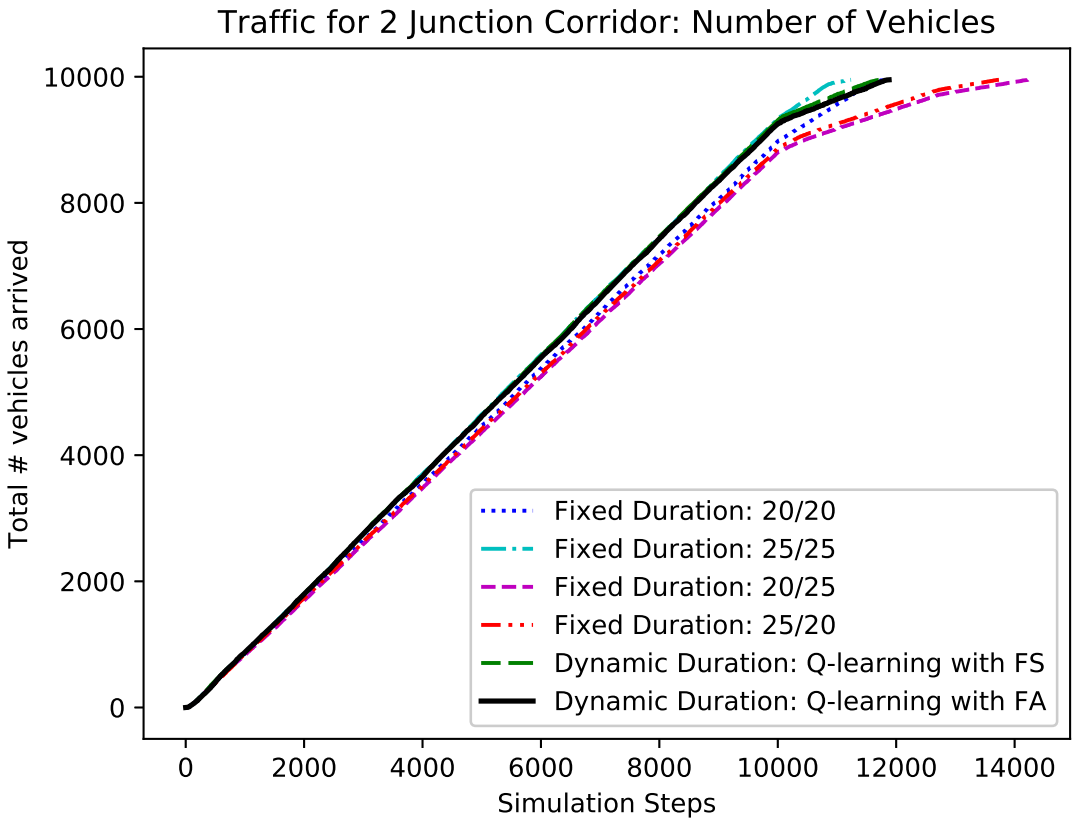


Figure 5.8: TNV for two-juncion corridor

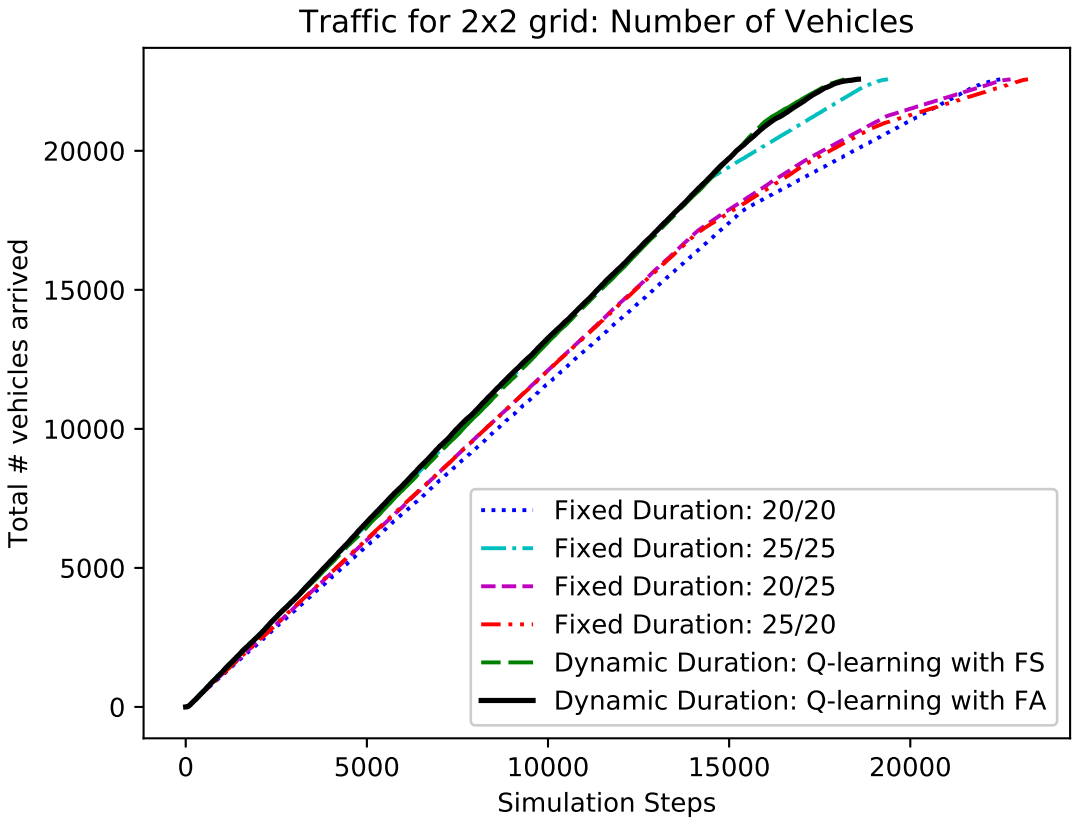


Figure 5.9: TNV for 2×2 grid

From the above figures, we observe that QLFS and QLFA algorithms outperform other algorithms in terms of number of vehicles reaching their destinations over the course of simulation.

This analysis shows that dynamic TSC algorithms can and do perform better than traditional fixed time TSC algorithms in reducing the average waiting time while maintaining high traffic flow, and that function approximation performs better while giving a performance boost due to its computational efficiency as compared to QLFS.

CHAPTER 6

Conclusions

Planning of a road traffic management policy dependent on sensors that accomplishes maximum traffic flux rates with least amount of congestion is difficult. RL offers a compelling worldview for tackling such issues. We have implemented three RL algorithms, and later evaluated them. So, concerning the simulation performances, it is witnessed that the dynamic duration algorithms perform better than the fixed duration ones overall and the function-approximation-based algorithm QLFA, performs better its the full-state-based counterpart, QLFS.

As future work, it would be interesting to investigate policy gradient methods for traffic signal control.

REFERENCES

1. **(DLR), G. A. C.** *et al.* (2001–2019). Simulation of urban mobility(sumo). URL http://sumo.dlr.de.

2. **Prashanth, L. A.** and **S. Bhatnagar** (2011). Reinforcement learning with function approximation for traffic signal control. *IEEE Transactions on Intelligent Transportation Systems*, **12**(2), 412–421.

3. **Sutton, R. S.** and **A. Barto**, *Reinforcement Learning: an Introduction*. The MIT Press, 2018, 2nd edition.

4. **Wiering, M.** and **M. van Otterlo**, *Reinforcement Learning: State-of-the-Art*. Springer Publishing Company, Incorporated, 2014. ISBN 364244685X, 9783642446856.